Do Exercício Acadêmico

↪ à Lógica de Automação Industrial

Disciplina: Algoritmos e Lógica de Programação **Instituição:** UniFECAF
Aluna: Bruna A. S. Pellegrini Rezer.

1. Contextualização do Desafio

Nas linhas de produção industriais, a inspeção manual de peças é um dos processos mais suscetíveis a falhas. Quando um operador precisa avaliar centenas de peças por turno, conferindo peso, cor e dimensões uma por uma, o risco de erros aumenta à medida que a fadiga chega e o custo operacional cresce na mesma proporção. Pequenas inconsistências que passam despercebidas podem comprometer lotes inteiros ou causar retrabalho desnecessário.

A automação digital resolve esse problema ao transferir a lógica de decisão para um sistema computacional. Critérios são definidos uma única vez e aplicados de forma consistente em todas as peças, sem exceção. Além de eliminar falhas humanas na conferência, o sistema registra cada decisão com data, hora e motivo. Criando um histórico rastreável que facilita auditorias e melhoria contínua do processo.

Este trabalho apresenta um protótipo desenvolvido em Python que simula exatamente essa lógica. Um sistema de inspeção e armazenamento de peças capaz de receber dados, aplicar critérios de qualidade, organizar aprovadas em caixas e gerar relatórios consolidados.

2. Estruturação do Raciocínio Lógico

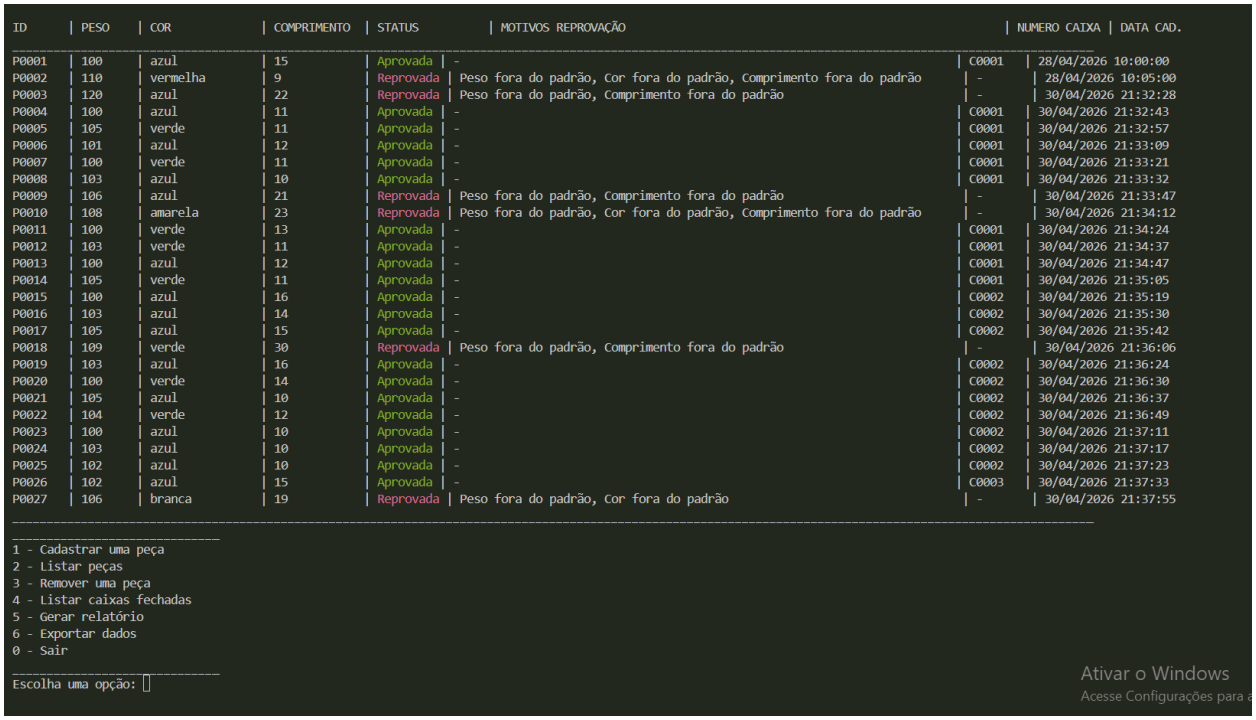
2.1 Decisões e Condições

A lógica central do sistema está na validação automática de cada peça cadastrada. Cada critério de qualidade é avaliado de forma independente, o que permite identificar com precisão quantos e quais critérios uma peça descumpriu:

```
if peso >= 95 and peso <= 105:
```

```
    nova_peca["status"] = "aprovada"
else:
    nova_peca["status"] = "reprovada"
    nova_peca["motivos_reprovacao"].append("Peso fora do padrão")
if cor.lower() not in ["azul", "verde"]:
    nova_peca["status"] = "reprovada"
    nova_peca["motivos_reprovacao"].append("Cor fora do padrão")
if comprimento < 10 or comprimento > 20:
    nova_peca["status"] = "reprovada"
    nova_peca["motivos_reprovacao"].append("Comprimento fora do padrão")
```

Essa abordagem permite que uma mesma peça acumule múltiplos motivos de reprovação. Algo que um processo manual tende a registrar de forma incompleta.



ID	PESO	COR	COMPRIMENTO	STATUS	MOTIVOS REPROVAÇÃO	NUMERO CAIXA	DATA CAD.
P0001	100	azul	15	Aprovada	-	C0001	28/04/2026 10:00:00
P0002	110	vermelha	9	Reprovada	Peso fora do padrão, Cor fora do padrão, Comprimento fora do padrão	-	28/04/2026 10:05:00
P0003	120	azul	22	Reprovada	Peso fora do padrão, Comprimento fora do padrão	-	30/04/2026 21:32:28
P0004	100	azul	11	Aprovada	-	C0001	30/04/2026 21:32:43
P0005	105	verde	11	Aprovada	-	C0001	30/04/2026 21:32:57
P0006	101	azul	12	Aprovada	-	C0001	30/04/2026 21:33:09
P0007	100	verde	11	Aprovada	-	C0001	30/04/2026 21:33:21
P0008	103	azul	10	Aprovada	-	C0001	30/04/2026 21:33:32
P0009	106	azul	21	Reprovada	Peso fora do padrão, Comprimento fora do padrão	-	30/04/2026 21:33:47
P0010	108	amarela	23	Reprovada	Peso fora do padrão, Cor fora do padrão, Comprimento fora do padrão	-	30/04/2026 21:34:12
P0011	100	verde	13	Aprovada	-	C0001	30/04/2026 21:34:24
P0012	103	verde	11	Aprovada	-	C0001	30/04/2026 21:34:37
P0013	100	azul	12	Aprovada	-	C0001	30/04/2026 21:34:47
P0014	105	verde	11	Aprovada	-	C0001	30/04/2026 21:35:05
P0015	100	azul	16	Aprovada	-	C0002	30/04/2026 21:35:19
P0016	103	azul	14	Aprovada	-	C0002	30/04/2026 21:35:30
P0017	105	azul	15	Aprovada	-	C0002	30/04/2026 21:35:42
P0018	109	verde	30	Reprovada	Peso fora do padrão, Comprimento fora do padrão	-	30/04/2026 21:36:06
P0019	103	azul	16	Aprovada	-	C0002	30/04/2026 21:36:24
P0020	100	verde	14	Aprovada	-	C0002	30/04/2026 21:36:30
P0021	105	azul	10	Aprovada	-	C0002	30/04/2026 21:36:37
P0022	104	verde	12	Aprovada	-	C0002	30/04/2026 21:36:49
P0023	100	azul	10	Aprovada	-	C0002	30/04/2026 21:37:11
P0024	103	azul	10	Aprovada	-	C0002	30/04/2026 21:37:17
P0025	102	azul	10	Aprovada	-	C0002	30/04/2026 21:37:23
P0026	102	azul	15	Aprovada	-	C0003	30/04/2026 21:37:33
P0027	106	branca	19	Reprovada	Peso fora do padrão, Cor fora do padrão	-	30/04/2026 21:37:55

```
1 - Cadastrar uma peça
2 - Listar peças
3 - Remover uma peça
4 - Listar caixas fechadas
5 - Gerar relatório
6 - Exportar dados
0 - Sair
Escolha uma opção:
```

Ativar o Windows
Acesse Configurações para a

print do terminal mostrando o cadastro de uma peça reprovada com múltiplos motivos.

2.2 Funções e Modularização

O programa é dividido em funções com responsabilidades bem definidas. Essa organização, chamada de modularização, é uma boa prática essencial em sistemas industriais: facilita manutenção, leitura do código e a possibilidade de expandir o sistema sem precisar reescrever tudo.

As principais funções e seus papéis:

Função	**Responsabilidade**
cadastrar_peca()	Coleta dados do operador, valida e registra a peça
armazenar_peca_em_caixa()	Organiza peças aprovadas em caixas de até 10 unidades
listar_pecas()	Exibe todas as peças em tabela formatada no terminal
remover_peca()	Permite excluir uma peça pelo ID
listar_caixas_fechadas()	Mostra caixas completas e suas respectivas peças
gerar_relatorio()	Consolida totais e motivos de reprovação no terminal
obter_dados_pecas()	Função auxiliar que padroniza os dados para exibição e exportação
exportar_dados()	Coordena a geração dos três arquivos de exportação

2.3 Repetição e Controle de Fluxo

O sistema usa dois tipos principais de repetição. O laço principal da aplicação mantém o menu ativo até o operador escolher sair:

```
def app():
```

```
while True:
    exibir_menu()
    escolha = input("Escolha uma opção: ")
    ...
    elif escolha == "0":
        break
```

Já nos formulários de entrada, laços “while True” com “try/except” garantem que o sistema não quebre caso o operador digite um valor inválido. Algo crítico em ambientes industriais onde o operador pode não ter familiaridade com terminais:

```
while True:
    try:
        peso = int(input("Digite o peso da peça: "))
        break
    except ValueError:
        print("Entrada inválida. Por favor, digite um número inteiro.")
```

3. Estruturas de Dados Utilizadas

O sistema representa as peças como dicionários dentro de uma lista. O que funciona como um banco de dados em memória. Cada peça possui os seguintes campos:

```
{
    "id": "P0001",
    "peso": 100,
    "cor": "azul",
```

"comprimento": 15,

"status": "aprovada",

"motivos_reprovacao": [],

"numero_caixa": "C0001",

"data_cadastro": "28/04/2026 10:00:00"

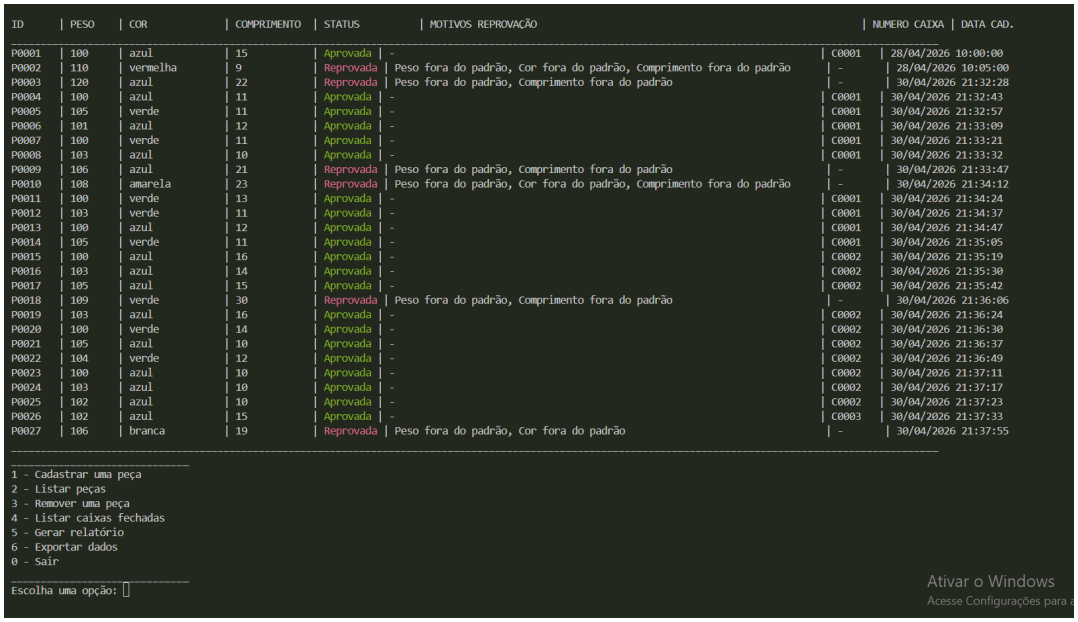
}

Lista (banco_de_dados): Armazena todas as peças cadastradas na sessão. Permite percorrer, filtrar, adicionar e remover peças de forma simples.

Dicionário: Cada peça é um conjunto de pares chave-valor, o que torna o acesso a qualquer informação direto e legível. Por exemplo: “peca["status"]” retorna imediatamente se a peça foi aprovada ou reprovada.

Lista dentro do dicionário (motivos_reprovacao): Cada peça carrega sua própria lista de motivos, o que permite múltiplos registros sem estruturas complexas.

As caixas seguem a mesma lógica: cada caixa é um dicionário com número e lista de peças, e todas as caixas fechadas ficam em “caixas_fechadas”.



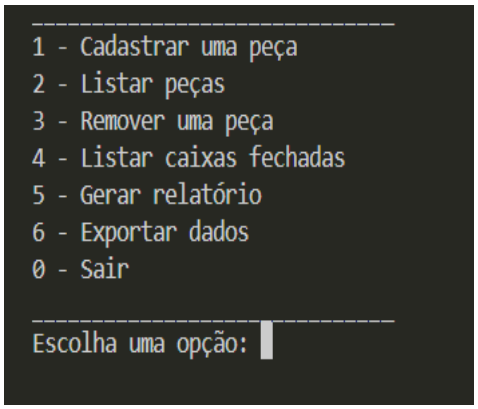
ID	PESO	COR	COMPRIMENTO	STATUS	MOTIVOS REPROVAÇÃO	NUMERO CAIXA	DATA CAD.
P0001	100	azul	15	Aprovada	-	C0001	28/04/2026 10:00:00
P0002	110	vermelha	9	Reprovada	Peso fora do padrão, Cor fora do padrão, Comprimento fora do padrão	-	28/04/2026 10:05:00
P0003	120	azul	22	Reprovada	Peso fora do padrão, Comprimento fora do padrão	-	30/04/2026 21:32:28
P0004	100	azul	11	Aprovada	-	C0001	30/04/2026 21:32:43
P0005	105	verde	11	Aprovada	-	C0001	30/04/2026 21:32:57
P0006	101	azul	12	Aprovada	-	C0001	30/04/2026 21:33:09
P0007	100	verde	11	Aprovada	-	C0001	30/04/2026 21:33:21
P0008	103	azul	10	Aprovada	-	C0001	30/04/2026 21:33:32
P0009	106	azul	21	Reprovada	Peso fora do padrão, Comprimento fora do padrão	-	30/04/2026 21:33:47
P0010	108	amarela	23	Reprovada	Peso fora do padrão, Cor fora do padrão, Comprimento fora do padrão	-	30/04/2026 21:34:12
P0011	100	verde	13	Aprovada	-	C0001	30/04/2026 21:34:24
P0012	103	verde	11	Aprovada	-	C0001	30/04/2026 21:34:37
P0013	100	azul	12	Aprovada	-	C0001	30/04/2026 21:34:47
P0014	105	verde	11	Aprovada	-	C0001	30/04/2026 21:35:05
P0015	100	azul	16	Aprovada	-	C0002	30/04/2026 21:35:19
P0016	103	azul	14	Aprovada	-	C0002	30/04/2026 21:35:30
P0017	105	azul	15	Aprovada	-	C0002	30/04/2026 21:35:42
P0018	109	verde	30	Reprovada	Peso fora do padrão, Comprimento fora do padrão	-	30/04/2026 21:36:06
P0019	103	azul	16	Aprovada	-	C0002	30/04/2026 21:36:24
P0020	100	verde	14	Aprovada	-	C0002	30/04/2026 21:36:30
P0021	105	azul	10	Aprovada	-	C0002	30/04/2026 21:36:37
P0022	104	verde	12	Aprovada	-	C0002	30/04/2026 21:36:49
P0023	100	azul	10	Aprovada	-	C0002	30/04/2026 21:37:11
P0024	103	azul	10	Aprovada	-	C0002	30/04/2026 21:37:17
P0025	102	azul	10	Aprovada	-	C0002	30/04/2026 21:37:23
P0026	102	azul	15	Aprovada	-	C0003	30/04/2026 21:37:33
P0027	106	branca	19	Reprovada	Peso fora do padrão, Cor fora do padrão	-	30/04/2026 21:37:55

Print do terminal mostrando a listagem de peças com tabela formatada.

4. Funcionalidades do Sistema

Menu Interativo

O sistema é operado por um menu de texto simples, acessível via terminal. Essa escolha é intencional, os terminais são leves, rodam em qualquer máquina e não dependem de interface gráfica. Ideal para protótipos industriais.



Print do menu principal no terminal.

Armazenamento em Caixas

Peças aprovadas são automaticamente inseridas na caixa atual. Quando a caixa atinge 10 peças, ela é fechada e uma nova é iniciada. Sem intervenção do operador. O número da caixa é gerado automaticamente no formato “C0001”,” C0002”, etc.

```
CAIXAS FECHADAS:

_____________________________________________
Caixa C0001
Peças: P0001 P0004 P0005 P0006 P0007 P0008 P0011 P0012 P0013 P0014

_____________________________________________
Caixa C0002
Peças: P0015 P0016 P0017 P0019 P0020 P0021 P0022 P0023 P0024 P0025

_____________________________________________
____________________________
1 - Cadastrar uma peça
2 - Listar peças
3 - Remover uma peça
4 - Listar caixas fechadas
5 - Gerar relatório
6 - Exportar dados
0 - Sair

_____________________________
Escolha uma opção:
```

Print da listagem de caixas fechadas.

Relatório no Terminal

O relatório consolida os números da sessão, total de peças, aprovadas, reprovadas, caixas utilizadas e frequência de cada motivo de reprovação.

```
RELATÓRIO DE PRODUÇÃO:

_____________________________________________________________
Total de peças produzidas: 27
Total de peças aprovadas: 21
Total de peças reprovadas: 6
Quantidade de caixas fechadas: 2
Quantidade total de caixas utilizadas (fechadas + atual): 3
Peças na caixa atual: 1

_____________________________________________________________
Motivos de reprovação:
Peso fora do padrão: 6 peças
Cor fora do padrão: 3 peças
Comprimento fora do padrão: 5 peças

_____________________________________________________________
____________________________
1 - Cadastrar uma peça
2 - Listar peças
3 - Remover uma peça
4 - Listar caixas fechadas
5 - Gerar relatório
6 - Exportar dados
0 - Sair

_____________________________
Escolha uma opção:
```

Print do relatório gerado no terminal.

Exportação de Dados

Ao acionar a opção 6, o sistema gera automaticamente três arquivos com timestamp no nome:

“pecas_auditadas_AAAAMMDD_HHMMSS.csv”: Com dados de todas as peças, compatível com Excel e Google Sheets.

“pecas_auditadas_AAAAMMDD_HHMMSS.txt” : Tabela formatada para viasualização no terminal + resumo de caixas.

“relatorio_producao_AAAAMMDD_HHMMSS.txt” : Relatório consolidado de produção.

O uso de timestamp no nome do arquivo impede que exportações se sobreponham, criando um histórico automático de cada sessão de inspeção.

Nome	Data de modificação	Tipo	Tamanho
codigo	30/04/2026 20:45	Arquivo Fonte Python	16 KB
pecas_auditadas_20260430_200524	30/04/2026 20:05	Arquivo Fonte Comma Separated Values	1 KB
pecas_auditadas_20260430_200524	30/04/2026 20:05	Documento de Texto	2 KB
pecas_auditadas_20260430_201603	30/04/2026 20:16	Arquivo Fonte Comma Separated Values	1 KB
pecas_auditadas_20260430_201903	30/04/2026 20:19	Arquivo Fonte Comma Separated Values	1 KB
pecas_auditadas_20260430_201903	30/04/2026 20:19	Documento de Texto	2 KB
pecas_auditadas_20260430_202243	30/04/2026 20:22	Arquivo Fonte Comma Separated Values	1 KB
pecas_auditadas_20260430_202243	30/04/2026 20:22	Documento de Texto	2 KB
pecas_auditadas_20260430_202457	30/04/2026 20:24	Arquivo Fonte Comma Separated Values	1 KB
pecas_auditadas_20260430_202457	30/04/2026 20:24	Documento de Texto	2 KB
pecas_auditadas_20260430_203235	30/04/2026 20:32	Arquivo Fonte Comma Separated Values	1 KB
pecas_auditadas_20260430_203235	30/04/2026 20:32	Documento de Texto	2 KB
pecas_auditadas_20260430_215015	30/04/2026 21:50	Arquivo Fonte Comma Separated Values	3 KB
pecas_auditadas_20260430_215015	30/04/2026 21:50	Documento de Texto	6 KB
relatorio_producao_2026-04-30_20-19-03	30/04/2026 20:19	Documento de Texto	1 KB
relatorio_producao_2026-04-30_20-22-43	30/04/2026 20:22	Documento de Texto	1 KB
relatorio_producao_2026-04-30_20-24-57	30/04/2026 20:24	Documento de Texto	1 KB
relatorio_producao_2026-04-30_20-32-35	30/04/2026 20:32	Documento de Texto	1 KB
relatorio_producao_2026-04-30_21-50-15	30/04/2026 21:50	Documento de Texto	1 KB
relatorio_producao_20260430_200524	30/04/2026 20:05	Documento de Texto	1 KB
tempCodeRunnerFile	30/04/2026 01:29	Arquivo Fonte Python	7 KB

Print dos arquivos gerados na pasta do projeto.

A1 | fx # Arquivo gerado em: 30/04/2026 21:50:15

	A	B	C	D	E	F	G	H	I
1	# Arquivo gerado em: 30/04/2026 21:50:15								
2	ID	PESO	COR	COMPRIMENTO	STATUS	MOTIVOS_REPROVACAO	NUMERO_CAIXA	DATA_CADASTRO	
3	P0001	100	azul	15	aprovada	-	C0001	28/04/2026 10:00:00	
4	P0002	110	vermelha	9	reprovada	Peso fora do padrão; Cor fora do padrão; Comprimento fora do padrão	-	28/04/2026 10:05:00	
5	P0003	120	azul	22	reprovada	Peso fora do padrão; Comprimento fora do padrão	-	30/04/2026 21:32:28	
6	P0004	100	azul	11	aprovada	-	C0001	30/04/2026 21:32:43	
7	P0005	105	verde	11	aprovada	-	C0001	30/04/2026 21:32:57	
8	P0006	101	azul	12	aprovada	-	C0001	30/04/2026 21:33:09	
9	P0007	100	verde	11	aprovada	-	C0001	30/04/2026 21:33:21	
10	P0008	103	azul	10	aprovada	-	C0001	30/04/2026 21:33:32	
11	P0009	106	azul	21	reprovada	Peso fora do padrão; Comprimento fora do padrão	-	30/04/2026 21:33:47	
12	P0010	108	amarela	23	reprovada	Peso fora do padrão; Cor fora do padrão; Comprimento fora do padrão	-	30/04/2026 21:34:12	
13	P0011	100	verde	13	aprovada	-	C0001	30/04/2026 21:34:24	
14	P0012	103	verde	11	aprovada	-	C0001	30/04/2026 21:34:37	
15	P0013	100	azul	12	aprovada	-	C0001	30/04/2026 21:34:47	
16	P0014	105	verde	11	aprovada	-	C0001	30/04/2026 21:35:05	
17	P0015	100	azul	16	aprovada	-	C0002	30/04/2026 21:35:19	
18	P0016	103	azul	14	aprovada	-	C0002	30/04/2026 21:35:30	
19	P0017	105	azul	15	aprovada	-	C0002	30/04/2026 21:35:42	
20	P0018	109	verde	30	reprovada	Peso fora do padrão; Comprimento fora do padrão	-	30/04/2026 21:36:06	
21	P0019	103	azul	16	aprovada	-	C0002	30/04/2026 21:36:24	
22	P0020	100	verde	14	aprovada	-	C0002	30/04/2026 21:36:30	
23	P0021	105	azul	10	aprovada	-	C0002	30/04/2026 21:36:37	
24	P0022	104	verde	12	aprovada	-	C0002	30/04/2026 21:36:49	
25	P0023	100	azul	10	aprovada	-	C0002	30/04/2026 21:37:11	
26	P0024	103	azul	10	aprovada	-	C0002	30/04/2026 21:37:17	
27	P0025	102	azul	10	aprovada	-	C0002	30/04/2026 21:37:23	
28	P0026	102	azul	15	aprovada	-	C0003	30/04/2026 21:37:33	
29	P0027	106	branca	19	reprovada	Peso fora do padrão; Cor fora do padrão	-	30/04/2026 21:37:55	
30									
31									
32									
33									

Print do conteúdo do CSV aberto no Google Sheets.

Arquivo Editar Exibir H1 B I

```
RELATÓRIO DE PRODUÇÃO
_____________________________________________________
Total de peças produzidas: 27
Total de peças aprovadas: 21
Total de peças reprovadas: 6
Quantidade de caixas fechadas: 2
Quantidade total de caixas utilizadas (fechadas + atual): 3
Peças na caixa atual: 1
_____________________________________________________
Motivos de reprovação:
Peso fora do padrão: 6 peça(s)
Cor fora do padrão: 3 peça(s)
Comprimento fora do padrão: 5 peça(s)
_____________________________________________________
```

Print do arquivo TXT de relatório.

5. Bibliotecas Utilizadas

O programa utiliza três bibliotecas para ampliar suas capacidades:

“os” : Detecta o sistema operacional e limpa o terminal entre operações, mantendo a interface limpa durante o uso contínuo.

“datetime” : Registra a data e hora exata do cadastro de cada peça e da geração dos arquivos exportados. Isso garante rastreabilidade temporal, um requisito básico em sistemas de controle de qualidade industrial.

"data_cadastro": datetime.now().strftime("%d/%m/%Y %H:%M:%S")

“colorama” : Adiciona cores ao terminal. Peças aprovadas aparecem em verde, reprovadas em vermelho. Em uma linha de produção onde o operador precisa identificar rapidamente o status de dezenas de itens, essa distinção visual reduz significativamente o tempo de leitura e o risco de confusão.

6. Benefícios Percebidos e Desafios Enfrentados

Benefícios

Consistência na inspeção: Os critérios são aplicados de forma idêntica em todas as peças, sem variação por fadiga ou interpretação.

Rastreabilidade: Cada peça registra data de cadastro, status e motivos. Isso permite recompor o histórico de qualquer lote.

Eficiência operacional: O fechamento automático de caixas elimina uma etapa manual e garante que nenhuma caixa ultrapasse a capacidade.

Exportação pronta para uso: Os arquivos CSV gerados podem ser abertos diretamente em planilhas sem nenhum tratamento adicional.

Desafios

Persistência em memória: Os dados existem apenas enquanto o programa está rodando. Uma interrupção inesperada (queda de energia, fechamento acidental) apaga todo o histórico da sessão. Isso foi contornado em parte pela exportação de dados, mas exigiria um banco de dados real numa versão de produção.

Interface exclusivamente textual: O terminal exige que o operador saiba digitar corretamente os dados. Em fábricas reais, é comum integrar o sistema a leitores de código de barras ou sensores. O que eliminaria erros de entrada.

Validação de entrada para cor: Como a cor é digitada via menu numérico, esse ponto está protegido. Porém, campos como peso e comprimento dependem do operador digitar valores coerentes com a realidade.

7. Caminhos para Expansão

Este protótipo demonstra que a lógica de um sistema de inspeção industrial pode ser implementada com estruturas básicas de programação. Mas em um cenário real, algumas evoluções seriam naturais:

Integração com sensores: Em vez de o operador digitar o peso manualmente, uma balança conectada via serial ou USB enviaria o dado automaticamente ao sistema, eliminando etapas e erros humanos. O mesmo vale para sensores de comprimento, usando visão computacional ou encoders e câmeras para classificação de cor.

Banco de dados persistente: Substituir a lista em memória por um banco de dados relaciona, como o SQLite ou PostgreSQL, garantiria que os dados sobrevivessem a reinicializações e permitiria consultas históricas complexas. Como por exemplo, identificar quais turnos apresentam maior índice de reprovação por cor.

Interface web ou painel de controle: Uma interface gráfica acessível por tablet no chão de fábrica tornaria o sistema operável por qualquer colaborador, sem necessidade de familiaridade com o terminal.

Inteligência Artificial: Com dados históricos acumulados, seria possível treinar modelos para prever quais combinações de variáveis tendem a gerar reprovação, permitindo ajustes preventivos no processo de produção antes que as peças sejam fabricadas fora do padrão.

8. Do Protótipo ao MES Industrial

8.1 O que é um MES e por que ele importa

Um **MES (Manufacturing Execution System)** é um sistema de gestão que opera em tempo real no chão de fábrica, conectando máquinas, operadores e dados de produção numa camada intermediária entre o processo físico e os sistemas de gestão empresarial (como o

ERP). Ele monitora ordens de produção, qualidade, consumo de materiais, paradas de máquina e eficiência operacional. Tudo de forma automatizada e rastreável.

O protótipo desenvolvido neste trabalho já implementa, em escala didática, o núcleo lógico de um módulo de controle de qualidade de um MES recebendo dados de peças, aplicando critérios de aceitação, organizando o armazenamento em lotes e gerando relatórios consolidados. A distância entre esse protótipo e um MES real é de infraestrutura, não de conceito.

8.2 Evolução Técnica: Passo a Passo

Etapa 1 - Persistência de dados

O primeiro limite do protótipo atual é que os dados existem apenas enquanto o programa está rodando. A primeira evolução natural seria substituir a lista “`banco_de_dados`” em memória por um banco de dados real.

Com SQLite nativo do Python, a mudança seria mínima em lógica mas representaria um salto enorme em confiabilidade:

```
import sqlite3

conn = sqlite3.connect("producao.db")

cursor = conn.cursor()

cursor.execute("""

    CREATE TABLE IF NOT EXISTS pecas (

        id TEXT PRIMARY KEY,

        peso INTEGER,

        cor TEXT,

        comprimento INTEGER,

        status TEXT,

        numero_caixa TEXT,

        data_cadastro TEXT
```

```
    )

""")
```

Isso permitiria que o sistema sobrevivesse a quedas de energia, gerasse histórico entre turnos e suportasse consultas como "quantas peças foram reprovadas por peso no turno da tarde dos últimos 30 dias".

Etapa 2 - Coleta automática via sensores (IIoT)

No protótipo, o operador digita peso e comprimento manualmente — o que introduz risco de erro e depende de disponibilidade humana. Num MES real, esses dados chegam automaticamente via sensores conectados:

- **Balança serial/USB**: Envia o peso diretamente ao sistema ao detectar uma peça posicionada.
- **Sensor de comprimento** (encoder ou visão computacional): Mede e transmite o valor sem interação humana.
- **Câmera com visão computacional**: Classifica a cor da peça usando bibliotecas como OpenCV.

Em Python, uma balança conectada via porta serial seria lida assim:

```
import serial

porta = serial.Serial("COM3", baudrate=9600)

peso_lido = float(porta.readline().decode().strip())
```

Esse dado chegaria ao sistema da mesma forma que hoje chega pela digitação, a função “`cadastrar_peca()`” receberia o valor, validaria e registraria normalmente. A lógica interna não precisaria mudar.

Etapa 3 — API e integração com ERP

Com o banco de dados persistente e a coleta automatizada, o próximo passo seria expor os dados via API REST, permitindo que outros sistemas da fábrica (ERP, dashboard de gestão, aplicativo do supervisor) acessem as informações em tempo real.

Com frameworks como **FastAPI** (Python), o endpoint de cadastro de peça seria algo como:

```
from fastapi import FastAPI
```

```
app = FastAPI()

@app.post("/pecas")

def cadastrar(peca: PecaModel):

    resultado = validar_e_registrar(peca)

    return resultado
```

Isso transforma o protótipo em um serviço de controle de qualidade que pode ser consumido por qualquer sistema da empresa, sem que os outros sistemas precisem saber como a lógica de validação funciona internamente.

Etapa 4 — Inteligência Artificial para predição

Com histórico acumulado de meses ou anos, é possível aplicar modelos de machine learning para identificar padrões que precedem reprovações em série. Por exemplo: Se o sistema percebe que peças com peso entre 103g e 105g cadastradas logo após a troca de turno têm 40% mais chance de serem reprovadas por comprimento nas horas seguintes, isso pode indicar uma folga mecânica que se manifesta com o aquecimento da máquina.

Ferramentas como “scikit-learn” (Python) permitem treinar esses modelos com os próprios dados gerados pelo sistema:

```
from sklearn.ensemble import RandomForestClassifier

modelo = RandomForestClassifier()

modelo.fit(X_historico, y_status)

predicao = modelo.predict([[peso_atual, comprimento_atual]])
```

Isso transforma o sistema de reativo que detecta defeitos depois que ocorrem, para preditivo alertando antes que o defeito se manifeste.

8.3 Posicionamento do Protótipo na Arquitetura MES

Um MES completo opera em camadas. O protótipo deste trabalho ocupa a camada de controle de qualidade e rastreabilidade, que é exatamente onde a lógica de inspeção, armazenamento em caixas e relatório de reprovações é executada:

Camada MES	**Função**	**Correspondência no Protótipo**
Coleta de dados (IIoT/sensores)	Captura automática de medições	`input()` / futuramente: `serial.read()`
Controle de qualidade	Validação por critérios e registro de motivos	`cadastrar_peca()` + validações
Rastreabilidade	Histórico por peça com data e lote	`banco_de_dados` com `data_cadastro` e `numero_caixa`
Gestão de lotes	Organização em caixas com fechamento automático	`armazenar_peca_em_caixa()`
Relatórios operacionais	Resumo de aprovações, reprovações e caixas	`gerar_relatorio()` + exportações
Integração ERP	Exportação estruturada para outros sistemas	Arquivos CSV e TXT gerados

O protótipo em Python não é apenas um exercício acadêmico. É uma implementação funcional das regras de negócio que qualquer módulo de qualidade de um MES real precisaria ter. A diferença entre o que foi construído aqui e um sistema industrial de verdade está na robustez da infraestrutura, banco de dados, sensores, APIs… Não na lógica que decide se uma peça está aprovada ou reprovada.

O que mais vemos neste exercício é que todos esses avanços partem exatamente da mesma base lógica construída aqui. Entrada de dados, validação por critérios, armazenamento estruturado e geração de relatórios. A complexidade muda. A lógica, não.

Trabalho redigido por Bruna Pellegrini Rezer para a disciplina de Algoritmos e Lógica de Programação, UniFECAF.